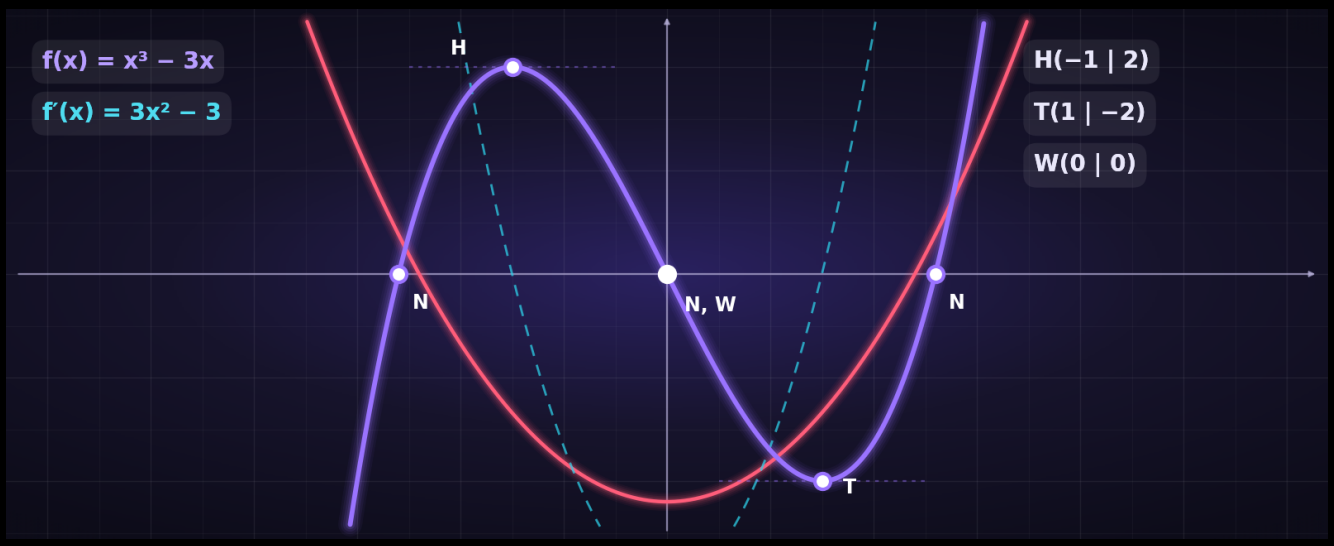


Project Documentation

Kurverafx

created by Daniel Kiebler

Android app for curve sketching of polynomial functions

Package de.kurverafx.app · Version 1.0 · Document version 1.0 · October 14, 2026

Table of Contents

1. Introduction 4

1.1 Purpose of the app 4

1.2 Key characteristics 4

1.3 Scope of this document 4

1.4 Glossary (German UI terms) 4

2. Requirements 6

2.1 Requirements for using the app 6

2.2 Functional requirements 6

2.3 Non-functional requirements 6

2.4 Requirements for development and modification 7

3. Features and Functions 8

3.1 User interface 8

3.2 Working with the app 8

3.3 Input syntax 8

3.4 Error messages 9

3.5 The analysis (six sections) 9

3.6 Graph markers 9

3.7 Worked examples 10

4. Technical Details 11

4.1 Technical facts 11

4.2 Architecture 11

4.3 APK structure 12

4.4 Native layer 12

4.5 Data model 12

4.6 Mathematical methods 12

4.7 Rendering pipeline 14

4.8 Interaction 14

4.9 Styling 14

4.10 Security, privacy and known limitations 15

5. Source Code Documentation 16

5.1 Overview of the source files 16

5.2 MainActivity.java 16

5.3 AndroidManifest.xml 17

5.4 assets/index.html – structure 17
5.5 Complete source listing – assets/index.html 19

6. Build, Test and Modify 29
6.1 Test the web part in a browser 29
6.2 Modify the existing APK 29
6.3 Rebuild as an Android Studio project 29
6.4 Manual test checklist 29
6.5 Roadmap ideas 29

Appendix A – Numerical constants 30

Appendix B – License and author 30

1. Introduction

1.1 Purpose of the app

Kurverafx is a small Android app for **curve sketching** (German: *Kurvendiskussion*) of polynomial functions. The user types a function such as `x^3 - 3x` and the app immediately draws its graph in an interactive coordinate system and produces a written, step-by-step analysis: the derivatives, the roots and the y-intercept, the local extrema, the inflection points and the intervals of monotonicity.

Curve sketching is a standard topic of upper-secondary mathematics, and a typical exercise consists of exactly these steps. Kurverafx automates the calculations and shows the calculation path next to the picture, so the result can be compared with a hand-written solution. Several functions can be shown at the same time, e.g. to see where two graphs intersect the axes or each other's region.

1.2 Key characteristics

Characteristic	Description
Type	Hybrid app: a minimal native Android shell (one *Activity* with a *WebView*) that displays a single local HTML page
Domain	Polynomial functions of degree 0 to 12 in the variable x
Offline	Fully offline. No network access, no permissions, no tracking, no accounts
Size	About 16 KB (APK), no third-party libraries
Interface language	German (e.g. “Nullstellen”, “Extremstellen”, “Wendestellen”); this documentation is in English, see glossary in 1.4
Platform	Android 8.0 (API 26) and newer
Author	Daniel Kiebler

1.3 Scope of this document

This document describes the app from three perspectives: what it does and which requirements it fulfils (chapters 2 and 3), how it works internally (chapter 4), and how the complete source code is structured (chapter 5). Chapter 6 explains how to build, test and modify the app. The complete source code of the app is listed in chapter 5; the same files are also part of the repository in the folder `source/`.

> **About the source code.** The web part (`assets/index.html`) is the original source of the app and is reproduced completely. The original file is minified; the listing in this document was formatted with Prettier for readability. The JavaScript is syntactically identical to the original (verified by comparing the parsed syntax trees); the only differences in CSS and HTML are cosmetic (trailing semicolons, 0.3px instead of .3px, self-closing slashes on void elements). The original comment is preserved. The Java part (`MainActivity`) and the manifest exist in the APK only in compiled form; they were **reconstructed from the bytecode and the binary manifest** and are marked as such.

1.4 Glossary (German UI terms)

German term in the app	English meaning
Kurvendiskussion	Curve sketching / full function analysis
Funktionen	Functions
Rechenweg & Kurvendiskussion	Calculation steps and curve analysis
Ausgangsfunktion	Original function
Ableitungen	Derivatives

German term in the app	English meaning
Achsenschnittpunkte	Axis intercepts
Nullstellen	Roots / zeros
y-Achsenabschnitt	y-intercept
Extremstellen	Extrema (candidates for maxima / minima)
Hochpunkt / Tiefpunkt	Local maximum / local minimum
Stationärer Punkt	Stationary point ($f' = 0$, but $f'' = 0$, so not classified)
Wendestellen / Wendepunkte	Inflection points
Monotonie	Monotonicity
Steigend / Fallend / Konstant	Increasing / decreasing / constant
Abszisse / Ordinate	x-axis / y-axis
Analyse	Analyse (button that selects a function for analysis)
Keine reellen Stellen	No real points

2. Requirements

2.1 Requirements for using the app

Item	Requirement
Operating system	Android 8.0 (API level 26) or newer (*minSdkVersion* 26; built for *targetSdkVersion* 36)
System component	Android System WebView must be installed and enabled (default on all standard devices)
WebView version	The page uses RegExp look-behind (`(?<!^)`) and `Array.prototype.at()`. The WebView therefore has to be based on Chromium 92 or newer. Since the WebView is updated through the app store, this is the case on current devices; very outdated, never-updated devices may not run the app
Storage	Approx. 16 KB
Network	Not required. The app declares no permissions, in particular no INTERNET permission
Input	Touchscreen (pan, pinch zoom) or mouse/keyboard (drag, wheel, Enter key)

2.2 Functional requirements

The following functional requirements were derived from the implemented behaviour of the app.

ID	Requirement
FR-01	The user can enter a polynomial function of degree 0–12 as text.
FR-02	Several functions can be managed at once; each gets its own colour (8-colour palette) and letter (f, g, h, …).
FR-03	All functions are drawn in one shared coordinate system with an adaptive grid and labelled axes.
FR-04	The view can be panned (drag) and zoomed (pinch, +/− buttons, mouse wheel) and reset to the default view.
FR-05	The app computes the first, second and third derivative.
FR-06	The app computes all real roots and the y-intercept.
FR-07	The app computes all stationary points and classifies them as local maximum, local minimum or stationary point using f″.
FR-08	The app computes inflection points (f″ = 0 with sign change of f″).
FR-09	The app determines the intervals in which the function is increasing or decreasing.
FR-10	Special points are marked on the graph (N, Y, H, T, S, W); the markers can be switched on and off.
FR-11	The calculation path (conditions, derivatives, candidates, tests) is shown as text for the selected function.
FR-12	Invalid input is rejected with an understandable error message; the existing functions are not changed.
FR-13	A function can be deleted from the list.
FR-14	The app works completely offline.

2.3 Non-functional requirements

ID	Requirement	How it is met
NFR-01	Privacy	No permissions, no network access, no analytics; cleartext traffic disabled
NFR-02	Small footprint	Single HTML file, no libraries, no images; APK of about 16 KB
NFR-03	Responsive layout	CSS grid with breakpoint at 850 px: one column on phones, two columns on tablets/desktop
NFR-04	Sharp rendering	Canvas is scaled by `devicePixelRatio` and redrawn on resize

ID	Requirement	How it is met
NFR-05	Touch operation	Pointer Events API, `touch-action: none` on the canvas, viewport zoom disabled
NFR-06	Readability	Dark UI, high-contrast white graph area, colour-coded functions
NFR-07	Maintainability	All logic in one self-contained file; no build step required for the web part

2.4 Requirements for development and modification

Task	Tools
Edit the app logic / UI	Any text editor; a desktop browser (Chrome, Edge, Firefox) to test `index.html` directly
Unpack / repack the APK	apktool, JDK 8 or newer
Sign the APK	zipalign and apksigner (Android SDK build-tools) plus a keystore; Android 7+ requires v2 signatures for installation
Install and debug	adb (Android platform-tools), USB debugging or an emulator (API 26+)
Rebuild as Android Studio project	Android Studio with an Empty Views Activity project, package `de.kurverafx.app`, minSdk 26

3. Features and Functions

3.1 User interface

The page consists of a header, the graph area with three round tool buttons, the “Functions” panel and the “Rechenweg & Kurvendiskussion” (calculation and analysis) panel. On screens narrower than 850 px everything is stacked in one column; on wider screens the graph and the function panel are placed side by side and the analysis panel spans the full width below.



Figure 1 – Schematic layout of the user interface (narrow and wide screens)

3.2 Working with the app

Action	How
Add a function	Type it in the input field and tap **+** or press Enter. The new function is selected automatically.
Analyse a function	Tap **Analyse** in its row; the analysis panel shows the six calculation sections.
Delete a function	Tap × in its row.
Move the view	Drag with one finger / the mouse.
Zoom	Pinch with two fingers (zoom is centred between the fingers), use the **+** / **−** buttons (factor 1.5) or the mouse wheel (factor 1.12 per step).
Reset the view	Tap ⌂: centre (0 | 0), 38 pixels per unit.
Show / hide markers	Checkbox “Nullstellen, Extrempunkte und Wendepunkte markieren”.

On first start two example functions are loaded: `f(x) = x³ − 3x` and `g(x) = x² − 4`. Initially the first function (f) is selected, so its analysis is shown right away.

3.3 Input syntax

Rule	Examples
Polynomial terms built from numbers, the lower-case variable x and non-negative integer powers	`2x^4 - 3x^2 + 5`
Power with ^ or with the superscripts ² ³ ⁴	`x^3`, `x³`, `2x²`

Rule	Examples
Coefficient 1 may be omitted	`x^2 - x`
Decimal point or decimal comma	`0.5x^2`, `0,5x^2`
An asterisk is ignored	`2*x^3`
Optional prefix `y=` or `f(x)=` (one letter)	`y = x^2 - 4`, `g(x)=x^3`
Spaces are ignored; the Unicode minus "−" is accepted	`x² − 4`
Constants	`5`
Terms with equal powers are added together	`x + x` → `2x`
Maximum degree 12	`x^12 + 1` is valid, `x^13` is rejected

Not supported: parentheses and products (`(x+1)(x-2)`), fractions, roots, trigonometric / exponential / logarithmic functions, upper-case `X`, negative or fractional exponents.

3.4 Error messages

Message in the app	Meaning	Typical cause
Bitte eine Funktion eingeben.	Please enter a function.	Empty input
Nur Polynome mit x, Zahlen und Potenzen eingeben.	Only enter polynomials using x, numbers and powers.	Characters other than 0–9 x + − . ^ (e.g. parentheses, letters)
Ungültiger Term: …	Invalid term: …	A term that does not match the pattern, e.g. `3x2` or `2.x`
Maximal Polynomgrad 12.	Maximum polynomial degree is 12.	A power greater than 12

3.5 The analysis (six sections)

#	Section (German heading)	Content
1	Ausgangsfunktion	The normalised function f(x)
2	Ableitungen	f′(x), f″(x), f‴(x)
3	Achsenschnittpunkte	Roots (solve f(x) = 0) and the y-intercept f(0)
4	Extremstellen	Necessary condition f′(x) = 0; candidates with type (Hochpunkt / Tiefpunkt / Stationärer Punkt) and the y-value; reminder of the f″ test
5	Wendestellen	Necessary condition f″(x) = 0 and sign-change check; the inflection points
6	Monotonie	Intervals between the extremum x-values with the sign of f′: increasing, decreasing or constant

Numbers are rounded to at most 5 decimal places and shown in German notation (decimal comma, e.g. 1,73205). In this document the decimal point is used.

3.6 Graph markers

Marker	Meaning	Position
N	Root (Nullstelle)	(x | 0)
Y	y-intercept	(0 | f(0))
H	Local maximum (Hochpunkt)	(x | f(x)) with f″(x) < 0
T	Local minimum (Tiefpunkt)	(x | f(x)) with f″(x) > 0

Marker	Meaning	Position
S	Stationary point (f′ = 0, f″ ≈ 0)	(x | f(x))
W	Inflection point (Wendepunkt)	(x | f(x))

3.7 Worked examples

The results below were produced by running the app's own functions (`parse`, `analysisData`) in Node.js.

Input	Roots	Extrema	Inflection points
`x^3 - 3x`	−1.73205; 0; 1.73205	H(−1 | 2), T(1 | −2)	W(0 | 0)
`x^2-4`	−2; 2	T(0 | −4)	none
`x^3-3x+1`	−1.87939; 0.3473; 1.53209	H(−1 | 3), T(1 | −1)	W(0 | 1)
`x^4-2x^2`	−1.41421; 0; 1.41421	T(−1 | −1), H(0 | 0), T(1 | −1)	W(±0.57735 | −0.55556)
`2x^4 - 3x² + 5`	none (real)	T(±0.86603 | 3.875), H(0 | 5)	W(±0.5 | 4.375)

Detailed result for `x^3 - 3x`: f′(x) = 3x² − 3, f″(x) = 6x, f‴(x) = 6. The sign of f′ is positive on (−∞; −1), negative on (−1; 1) and positive on (1; +∞); the function is therefore increasing, decreasing, increasing.

4. Technical Details

4.1 Technical facts

Property	Value
App name	Kurverafx (string resource `app_name`)
Package name	`de.kurverafx.app`
Version	1.0 (versionCode 1)
minSdkVersion / targetSdkVersion	26 (Android 8.0) / 36
Launcher activity	`de.kurverafx.app.MainActivity` (exported, MAIN / LAUNCHER)
Permissions	none
Cleartext traffic	disabled (`usesCleartextTraffic="false"`)
Manifest build attributes	platformBuildVersionCode 23, platformBuildVersionName 6.0.1
APK signature	signed (v1 JAR signature `META-INF/KURVERAF.*` and APK Signature Scheme v2/v3 block)
Languages / technologies	Java (Activity + WebView), HTML5, CSS3, JavaScript (ES2022 features: look-behind RegExp, `Array.at`)
Third-party libraries	none
Rendering	HTML5 Canvas 2D
Approximate size	16 KB (index.html 13,986 bytes, classes.dex 1,432 bytes)

4.2 Architecture

Kurverafx has two layers. The **native layer** is a single Activity that creates a WebView, sets its background colour and a few settings and loads the local page. The **web layer** is one self-contained HTML file that contains the user interface, the parser, the mathematics and the renderer. There is no communication between the layers (no JavaScript interface, no WebViewClient), and no network requests are made.

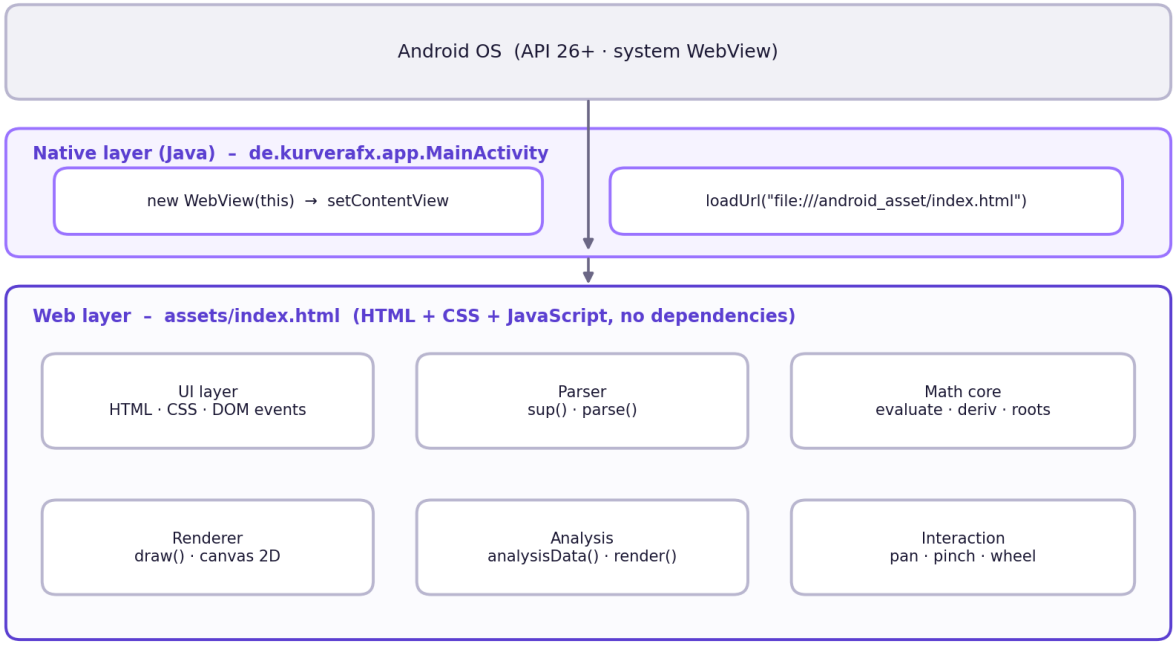


Figure 2 – Layered architecture of Kurverafx

4.3 APK structure

Entry	Size (bytes)	Purpose
`AndroidManifest.xml`	1,772	Binary manifest: package, SDK levels, launcher activity
`classes.dex`	1,432	Compiled `MainActivity` (Dalvik bytecode)
`resources.arsc`	580	Compiled resources: string `app_name` = “Kurverafx”
`assets/index.html`	13,986	The complete application (UI, logic, rendering)
`META-INF/*`	≈ 2,100	Signature files: MANIFEST.MF, KURVERAF.SF, KURVERAF.RSA

4.4 Native layer

`MainActivity` extends `android.app.Activity`. In `onCreate` it creates a `WebView`, makes it the content view, sets the background colour `#100F1E` (identical to the page background, which avoids a white flash during loading), configures the settings and loads `file:///android_asset/index.html`. In `onDestroy` the WebView is destroyed to release its resources.

WebView setting	Value	Reason
setJavaScriptEnabled	true	The whole app is written in JavaScript
setAllowFileAccess	true	Page is loaded from the local asset folder
setDomStorageEnabled	false	No localStorage / sessionStorage needed; nothing is stored
setBuiltInZoomControls	false	The app implements its own zoom; page zoom is also disabled in the viewport meta tag

No `addJavascriptInterface` is used, so no native object is exposed to the page. The code does not save any instance state (see 4.10).

4.5 Data model

A polynomial is stored as a plain array of coefficients indexed by the power of x: `c[i]` is the coefficient of x^i. Example: `x^3 - 3x` → `[0, -3, 0, 1]`. The array has no trailing zeros (except for the zero polynomial `[0]`). All algorithms operate on this array.

State variable	Type	Meaning
`funcs`	Array<{c, color}>	The functions: coefficient array and display colour
`selected`	number	Index of the function shown in the analysis panel
`view`	{cx, cy, scale}	World coordinates of the view centre and pixels per unit (default 0, 0, 38)
`pointers`	Map	Active pointers (touches) with their client coordinates
`palette`	string[8]	Function colours, cycled with the function index

4.6 Mathematical methods

All computations use double-precision floating-point numbers.

4.6.1 Parsing

- **Normalisation** (`sup`): ² ³ ⁴ become `^2 ^3 ^4`; the Unicode minus becomes `-`; commas become points; whitespace and `*` are removed.
- **Prefix**: a leading `<letter>(x)=` or `y=` is removed. An empty rest raises “Bitte eine Funktion eingeben.”
- **Character check**: only `0-9 x + - . ^` are allowed.

- **Term splitting**: a separator is inserted in front of every `+` or `-` that is not the first character; each term must match `[sign][number][x[^power]]` and contain a number or an x.
- **Accumulation**: the coefficient of each power is added; powers above 12 are rejected; trailing zero coefficients ($|a| < 10^{-12}$) are removed.

4.6.2 Evaluation and derivatives

Horner scheme evaluates the polynomial with n multiplications: $y = (\ldots((a_n \cdot x + a_{n-1}) \cdot x + a_{n-2}) \cdot x + \ldots) \cdot x + a_0$. The **derivative** of $\Sigma\, a_i x^i$ is $\Sigma\, i \cdot a_i x^{i-1}$; the app derives up to three times. The derivative of a constant is the zero polynomial.

4.6.3 Real roots by recursive derivative isolation

A polynomial can change its sign only once between two consecutive roots of its derivative. The function `roots(c)` uses this to isolate every real root in an interval and then refines it by bisection:

- Degree 0 → no roots; degree 1 → $x = -a_0 / a_1$.
- Otherwise compute the critical points as the roots of the derivative (recursion).
- Compute a root bound $B = 1 + \max_{i<n} |a_i| / |a_n|$, limited to $[1, 10^7]$. All real roots lie in $[-B, B]$.
- The points $-B$, the critical points inside $(-B, B)$, and $+B$ divide the line into intervals in which f is monotonic.
- In each interval with f(left) · f(right) < 0 exactly one root exists; it is found by bisection (at most 90 iterations, stop when the interval is shorter than 10^{-10} or $|f| < 10^{-12}$).
- Critical points with $|f(x)| < 10^{-6} \cdot \max(1, |a_n|)$ are added as roots. This detects double (even-multiplicity) roots where f touches the axis without changing its sign.
- The result is sorted and duplicates closer than 10^{-5} are removed (`unique`).

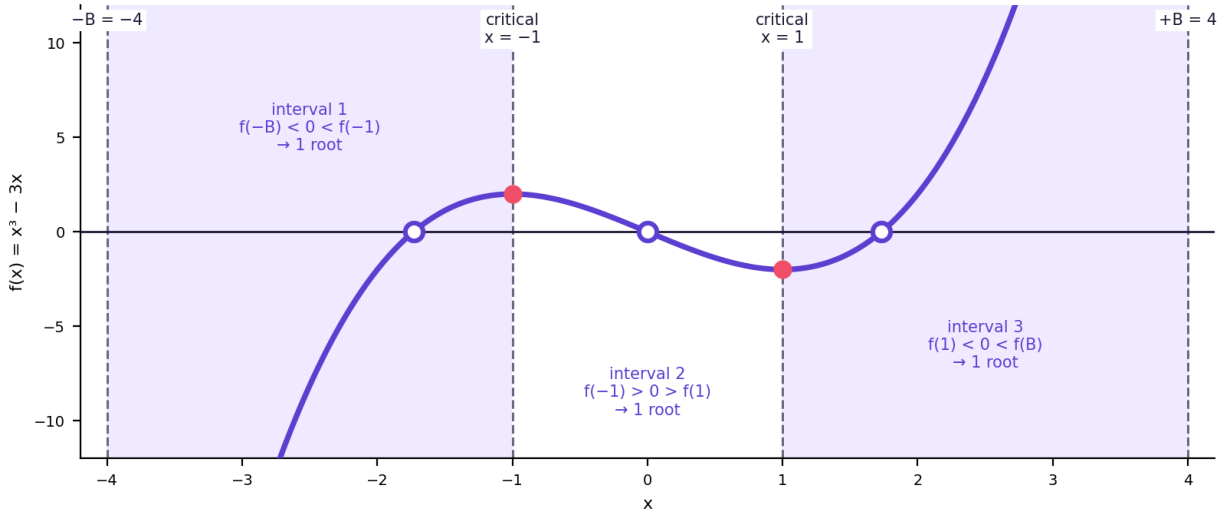


Figure 3 – Root isolation for f(x) = x³ − 3x: critical points split [−B, B] into three monotonic intervals, each with one sign change

4.6.4 Extrema

Candidates are the roots of f′ (necessary condition f′(x) = 0). The second derivative classifies them: $f''(x) > 10^{-7}$ → **local minimum** (Tiefpunkt); $f''(x) < -10^{-7}$ → **local maximum** (Hochpunkt); otherwise **stationary point** (further checking needed).

4.6.5 Inflection points

Candidates are the roots of f″ (necessary condition). A candidate x is accepted only if f″ changes its sign there, tested at x − h and x + h with $h = \max(10^{-4}, |x| \cdot 10^{-5})$. This excludes points such as x = 0 for $f(x) = x^4$.

4.6.6 Monotonicity

The real line is split at the x-values of the extrema. For each interval a test point is chosen (the midpoint for finite intervals, for an unbounded interval a point max(1, 10 % of |bound|) away from the finite end, and 0 if there are no extrema) and the sign of f′ is evaluated there: $> 10^{-8}$ → increasing, $< -10^{-8}$ → decreasing, otherwise constant.

4.7 Rendering pipeline

- **Coordinate transform:** with canvas width w, height h and view {cx, cy, scale}: X(x) = w/2 + (x − cx)·scale and Y(y) = h/2 − (y − cy)·scale; the inverse functions are `invX` and `invY`.
- **HiDPI:** the canvas pixel size is set to the CSS size times `devicePixelRatio` and the context is scaled accordingly on every redraw.
- **Adaptive grid:** the base step is $10^{\text{floor}(\log_{10}(85\ /\ \text{scale}))}$; if the resulting spacing is below 45 pixels it is doubled, and if it is still below 45 pixels it is multiplied by 2.5 (resulting in a 1–2–5 sequence). Labels are drawn at the axes, kept inside the visible area.
- **Axes:** drawn in dark slate, labelled “Ordinate” and “Abszisse”.
- **Curves:** sampled every 1.5 px. A segment is broken if the value is not finite, exceeds 10^7 pixels, or jumps by more than twice the canvas height from the previous sample (prevents vertical artefacts for steep curves). The selected function is drawn with a line width of 2.8 instead of 2.2.
- **Markers:** for each function the roots, the y-intercept, the extrema and the inflection points are drawn as white circles (radius 4.5) with a coloured border and a letter, only if they lie inside the canvas.

4.8 Interaction

Input	Implementation
One pointer moves	Pan: `cx -= dx / scale`, `cy += dy / scale`
Two pointers move	Pinch zoom: scale multiplied by the ratio of the finger distances; the world point under the finger midpoint stays fixed
Mouse wheel	scale × 1.12 (wheel up) or ÷ 1.12 (wheel down); default scrolling is prevented
Buttons + / − / ⌂	scale × 1.5, ÷ 1.5, reset to {0, 0, 38}
Zoom limits	scale clamped to 0.001 … 100,000 pixels per unit
Window resize	full redraw

4.9 Styling

The visual design uses a dark theme with purple accents and a white graph area. Colours are fixed values in the stylesheet:

Element	Colour
Page background / header	#100f1e / #1b1834
Panels (background / border)	#1d1a34 / #35304f
Primary button / accent	#815cf6
Analysis headings	#bba6ff
Error text	#ff9fa9
Graph area	#ffffff (grid #e7e7ed, axes #41415c)
Function palette	#7948e8, #f04e68, #008eaa, #d58a00, #258450, #a53eaf, #c75328, #4169e1

The layout switches at a viewport width of 850 px: two-column grid (1.4fr : 1fr) and a taller canvas (560 px instead of 390 px).

4.10 Security, privacy and known limitations

Security and privacy

- No permissions, no network, no analytics, no accounts; all calculations are local.
- No native bridge (`addJavascriptInterface`) is exposed to the page.
- User input is inserted into the page via `textContent` / `replaceChildren`, never as HTML, and the parser only accepts a small whitelist of characters; HTML injection through the input field is therefore not possible.
- The page contains no external resources (scripts, fonts, images).

Known limitations and observations

- **Polynomials only**, degree ≤ 12, no parentheses; real roots only.
- **Even-order stationary points** with $f'' = 0$ are reported as “Stationärer Punkt” and are not classified. Example: for `x^4` the point (0 | 0) is a minimum, but is shown as stationary (marker S). The same applies to the minimum of x^6.
- **Numerical accuracy:** results are approximations (tolerances in Appendix A); polynomials with very large coefficients or closely spaced roots may be inaccurate.
- **State is not persisted:** DOM storage is disabled, no state is saved, and the manifest declares no `configChanges`. A screen rotation therefore recreates the Activity and reloads the page, which resets the function list to the two example functions.
- **No back-button handling or WebViewClient:** the system default applies (back closes the Activity).
- **Not included:** symmetry, end behaviour / limits, concavity intervals, tangent lines, value table, result export.
- **UI language:** German only.
- **Unused variable:** `gesture` is declared and reset in the pointerdown handler but not used otherwise.

5. Source Code Documentation

5.1 Overview of the source files

File	Language	Role	Provenance
source/index.html	HTML / CSS / JS	Complete UI, parser, mathematics, renderer, interaction	Original source (minified, 13,986 bytes)
source/MainActivity.java	Java	Android entry point; hosts the WebView	Reconstructed from classes.dex
source/AndroidManifest.xml	XML	App metadata, launcher activity	Reconstructed from the binary manifest

5.2 MainActivity.java

Full source (reconstructed from the bytecode; the bytecode contains exactly the calls shown):

```java
// NOTE: Reconstructed from classes.dex (Dalvik bytecode) of Kurverafx 1.0.
// Behaviour is identical to the bytecode; comments and formatting were added.
package de.kurverafx.app;

import android.app.Activity;
import android.os.Bundle;
import android.webkit.WebSettings;
import android.webkit.WebView;

public class MainActivity extends Activity {

    private WebView web;

    @Override
    public void onCreate(Bundle savedInstanceState) {
        super.onCreate(savedInstanceState);

        web = new WebView(this);
        setContentView(web);

        // -15724770 == 0xFF100F1E, same colour as the page background (#100f1e)
        web.setBackgroundColor(0xFF100F1E);

        WebSettings settings = web.getSettings();
        settings.setJavaScriptEnabled(true);
        settings.setAllowFileAccess(true);
        settings.setDomStorageEnabled(false);
        settings.setBuiltInZoomControls(false);

        web.loadUrl("file:///android_asset/index.html");
    }

    @Override
    public void onDestroy() {
        web.destroy();
        super.onDestroy();
    }
}
```

Lines	Explanation
3–8	Package declaration and imports (Activity, Bundle, WebSettings, WebView).
10–12	The class extends Activity (not AppCompatActivity: no AndroidX dependency, hence the tiny APK). The field web holds the WebView.
15–19	onCreate calls the super implementation, creates the WebView and sets it as the only content view (full-screen web page).
21–22	Background colour 0xFF100F1E, identical to the CSS page background, avoids a white flash while the page loads.

Lines	Explanation
24–28	WebView settings: JavaScript on, file access on, DOM storage off, built-in zoom controls off.
30	Loads the local page from the APK assets.
34–36	`onDestroy` destroys the WebView before calling the super implementation.

5.3 AndroidManifest.xml

Reconstructed from the binary manifest of the APK:

```xml
<?xml version="1.0" encoding="utf-8"?>
<!-- Reconstructed from the binary manifest of Kurverafx 1.0 -->
<manifest xmlns:android="http://schemas.android.com/apk/res/android"
    package="de.kurverafx.app"
    android:versionCode="1"
    android:versionName="1.0">

    <uses-sdk
        android:minSdkVersion="26"
        android:targetSdkVersion="36" />

    <!-- no <uses-permission> elements: the app requests no permissions -->

    <application
        android:label="@string/app_name"
        android:usesCleartextTraffic="false">
        <!-- android:theme references the framework style with resource id 0x01030241 -->

        <activity
            android:name=".MainActivity"
            android:exported="true">
            <intent-filter>
                <action android:name="android.intent.action.MAIN" />
                <category android:name="android.intent.category.LAUNCHER" />
            </intent-filter>
        </activity>
    </application>
</manifest>
```

Element / attribute	Meaning
`package="de.kurverafx.app"`	Application ID and namespace of the Java class
`versionCode / versionName`	1 / 1.0
`uses-sdk`	minSdkVersion 26 (Android 8.0), targetSdkVersion 36
`android:label`	Reference to the string resource `app_name` = “Kurverafx”
`android:usesCleartextTraffic="false"`	HTTP traffic is forbidden (the app does not use the network anyway)
`activity .MainActivity`	Exported launcher activity with the MAIN action and LAUNCHER category
no *uses-permission*	The app requests no permissions

5.4 assets/index.html – structure

The file has three parts: the **style sheet** (lines 11–207), the **markup** (lines 209–251) and the **script** (lines 251–715). Line numbers refer to the formatted listing in 5.5.

5.4.1 Markup

Element	Purpose
`<header>`	Title “◈ Kurverafx” and the subtitle “Kurvendiskussion · Offline”
`.graphbox`	Container of the canvas `#graph`, the tool buttons (`#zin, #zout, #reset`) and the gesture hint

Element	Purpose
section.panel	Functions panel: input #expr, add button #add, error line #err, list #functions, checkbox #markers, syntax note
section.analysis-panel	Analysis: legend line and the container #analysis that is filled by render()

5.4.2 Style sheet (main classes)

Class / selector	Purpose
:root	System font stack, base text colour and page background
header	Dark title bar (flex layout)
.main	Centred content column, max. width 1100 px
.graphbox, canvas	Rounded white graph area; canvas 390 px high (560 px on wide screens), touch-action: none
.tools, .tools button	Round zoom / reset buttons placed top right over the graph
.hint	Gesture hint at the bottom left of the graph
.panel	Dark rounded card used for the function and analysis panels
.entry, .primary	Input row and the purple add button
.item, .dot, .fn, .small, .selected	One row of the function list: colour dot, formula text, small buttons, highlight of the selected row
.analysis, .analysis h3	Analysis text and section headings
.error	Red error line
.layout	Single column; at ≥ 850 px two columns (1.4fr / 1fr) via media query

5.4.3 Script – functions

Name	Signature	Description	Returns	Line
$	$(id)	Shortcut for document.getElementById.	Element	252
sup	sup(s)	Normalises the input text: ² ³ ⁴ → ^2 ^3 ^4, “−” → “-”, “,” → “.”, removes whitespace and “*”.	string	270
parse	parse(src)	Parses the input into a coefficient array; validates characters, terms and degree; throws Error with a German message.	number[]	280
evaluate	evaluate(c, x)	Evaluates the polynomial with the Horner scheme.	number	304
deriv	deriv(c)	Derivative of the coefficient array; [0] for constants.	number[]	309
fmt	fmt(v)	Formats a number: “–” if not finite, tiny values → 0, max. 5 decimals, German locale.	string	312
poly	poly(c)	Renders a coefficient array as text, e.g. x^3 - 3x; skips zero terms, omits coefficient 1.	string	317
bisect	bisect(c, a, b)	Bisection on an interval with a sign change (≤ 90 iterations).	number	332
unique	unique(a)	Filters non-finite values, sorts ascending and removes near-duplicates (< 10^{-5}).	number[]	346
roots	roots(c)	All real roots by recursive derivative isolation, bisection and double-root detection.	number[]	353
analysisData	analysisData(c)	Derivatives, roots, extrema (with type) and inflection points of a polynomial.	{d, dd, ddd, zeros, ext, inf}	374

Name	Signature	Description	Returns	Line
`draw`	`draw()`	Repaints the canvas: background, adaptive grid, axes, curves, markers. Contains the closures `X, Y, invX, invY`.	void	397
`render`	`render()`	Rebuilds the function list and the six analysis sections, then calls `draw()`. Also registers the Analyse / × button handlers.	void	512
`distance`	`distance(a, b)`	Euclidean distance of two pointer positions (for pinch zoom).	number	657

5.4.4 Script – event handlers and initialisation

Target	Event	Action	Line
`#add`	click	Parses the input; on success adds the function with the next palette colour, selects it, clears input and error, calls `render()`; on failure shows the error text.	629
`#expr`	keydown	Enter triggers the add button.	641
`#markers`	change	Redraws the graph with / without markers.	644
`#zin / #zout`	click	scale × 1.5 / ÷ 1.5 (clamped), redraw.	645
`#reset`	click	view = {0, 0, 38}, redraw.	653
`canvas`	pointerdown	Captures the pointer, stores its position.	660
`canvas`	pointermove	Pan with one pointer, pinch zoom with two; redraw.	665
`canvas`	pointerup / pointercancel / lostpointercapture	Removes the pointer from the map.	695
`canvas`	wheel	Zoom by 1.12; prevents page scrolling (non-passive listener).	698
`window`	resize	Redraw.	709
`Analyse / × buttons`	click	Select a function for analysis / delete it (created inside `render()`).	527

The script ends with the initialisation `funcs = [x³ − 3x, x² − 4]; render();` (line 263), which shows the two example functions and the analysis of the first one.

5.5 Complete source listing – assets/index.html

The listing below is the complete file, formatted with Prettier (717 lines) and syntax-highlighted. The JavaScript is syntactically identical to the original minified file; CSS and HTML differ only cosmetically (see 1.3).

```
<!doctype html>
<html lang="de">
  <head>
    <meta charset="utf-8" />
    <meta
      name="viewport"
      content="width=device-width,initial-scale=1,maximum-scale=1,user-scalable=no"
    />
    <meta name="theme-color" content="#15132a" />
    <title>Kurverafx</title>
    <style>
      :root {
        font-family: system-ui, Arial, sans-serif;
        color: #e9e7fa;
        background: #100f1e;
      }
      * {
        box-sizing: border-box;
      }
```

```
      body {
        margin: 0;
      }
      header {
        background: #1b1834;
        padding: 16px 18px;
        display: flex;
        align-items: center;
        justify-content: space-between;
        border-bottom: 1px solid #3d375f;
      }
      header strong {
        font-size: 21px;
        letter-spacing: 0.3px;
      }
      header span {
        font-size: 12px;
        color: #b6afd8;
      }
      .main {
        max-width: 1100px;
        margin: auto;
        padding: 12px;
      }
      .graphbox {
        position: relative;
        border: 1px solid #514775;
        border-radius: 18px;
        overflow: hidden;
        background: #fff;
        touch-action: none;
      }
      canvas {
        width: 100%;
        height: 390px;
        display: block;
        touch-action: none;
      }
      .tools {
        position: absolute;
        right: 10px;
        top: 10px;
        display: grid;
        gap: 7px;
      }
      .tools button {
        background: #fff;
        color: #35246d;
        border: 1px solid #d6d0eb;
        box-shadow: 0 2px 6px #0002;
        border-radius: 10px;
        width: 38px;
        height: 38px;
        font-size: 21px;
      }
      .hint {
        position: absolute;
        bottom: 8px;
        left: 10px;
        color: #666;
        font-size: 11px;
        background: #ffffffd9;
        border-radius: 6px;
        padding: 4px;
      }
      .panel {
        background: #1d1a34;
        border: 1px solid #35304f;
        border-radius: 16px;
        padding: 15px;
        margin-top: 13px;
      }
      h2 {
        font-size: 16px;
        margin: 0 0 12px;
      }
      .entry {
```

```
        display: flex;
        gap: 7px;
      }
      .entry input {
        min-width: 0;
        flex: 1;
        background: #100f21;
        color: white;
        border: 1px solid #6a5c9d;
        border-radius: 9px;
        padding: 12px;
        font-size: 16px;
      }
      .primary {
        background: #815cf6;
        color: white;
        border: 0;
        border-radius: 9px;
        padding: 10px 17px;
        font-size: 20px;
      }
      .item {
        display: flex;
        align-items: center;
        gap: 9px;
        border-top: 1px solid #38324e;
        padding: 11px 0;
      }
      .dot {
        width: 13px;
        height: 13px;
        border-radius: 50%;
        flex: none;
      }
      .fn {
        flex: 1;
        min-width: 0;
        overflow-wrap: anywhere;
      }
      .small {
        background: #302b4b;
        color: #e8e3ff;
        border: 1px solid #524a72;
        border-radius: 7px;
        padding: 7px 9px;
      }
      .selected {
        outline: 1px solid #bca6ff;
      }
      .note {
        color: #afa9c7;
        font-size: 12px;
        margin: 10px 0 0;
      }
      .analysis {
        line-height: 1.6;
        font-size: 14px;
      }
      .analysis h3 {
        font-size: 15px;
        color: #bba6ff;
        margin: 15px 0 4px;
      }
      .analysis p {
        margin: 3px 0;
      }
      .analysis code {
        color: #eee2ff;
        background: #302b4c;
        padding: 2px 5px;
        border-radius: 4px;
      }
      .error {
        color: #ff9fa9;
        font-size: 13px;
        padding-top: 8px;
      }
      .legend {
```

```
        font-size: 12px;
        color: #afa9c7;
      }
      label {
        font-size: 13px;
        color: #d4cdea;
      }
      .switch {
        display: flex;
        gap: 10px;
        align-items: center;
        margin-top: 10px;
      }
      input[type="checkbox"] {
        accent-color: #946aff;
      }
      @media (min-width: 850px) {
        .layout {
          display: grid;
          grid-template-columns: 1.4fr 1fr;
          gap: 14px;
        }
        .panel {
          margin-top: 0;
        }
        canvas {
          height: 560px;
        }
        .analysis-panel {
          margin-top: 14px;
        }
      }
    </style>
  </head>
  <body>
    <header><strong>◈ Kurverafx</strong><span>Kurvendiskussion · Offline</span></header>
    <div class="main">
      <div class="layout">
        <div>
          <div class="graphbox">
            <canvas id="graph"></canvas>
            <div class="tools">
              <button id="zin" aria-label="Vergrößern">+</button
              ><button id="zout" aria-label="Verkleinern">−</button
              ><button id="reset" aria-label="Ansicht zurücksetzen">⌂</button>
            </div>
            <div class="hint">Mit 2 Fingern zoomen · Ziehen zum Verschieben</div>
          </div>
        </div>
        <section class="panel">
          <h2>Funktionen</h2>
          <div class="entry">
            <input
              id="expr"
              placeholder="z. B. x^3 - 3x"
              aria-label="Polynom eingeben"
            /><button class="primary" id="add" aria-label="Funktion hinzufügen">+</button>
          </div>
          <div id="err" class="error"></div>
          <div id="functions"></div>
          <div class="switch">
            <input type="checkbox" id="markers" checked /><label for="markers"
              >Nullstellen, Extrempunkte und Wendepunkte markieren</label
            >
          </div>
          <p class="note">
            Erlaubt: ganzrationale Funktionen, z. B. 2x^4 - 3x² + 5. Maximal Grad 12.
          </p>
        </section>
      </div>
      <section class="panel analysis-panel">
        <h2>Rechenweg & Kurvendiskussion</h2>
        <div class="legend">Wähle eine Funktion aus, um die Analyse anzuzeigen.</div>
        <div id="analysis" class="analysis"></div>
      </section>
    </div>
    <script>
      const $ = (id) => document.getElementById(id),
```

```
        palette = [
          "#7948e8",
          "#f04e68",
          "#008eaa",
          "#d58a00",
          "#258450",
          "#a53eaf",
          "#c75328",
          "#4169e1",
        ];
      let funcs = [],
        selected = 0,
        view = { cx: 0, cy: 0, scale: 38 },
        pointers = new Map(),
        gesture = null;
      const canvas = $("graph"),
        ctx = canvas.getContext("2d");
      function sup(s) {
        return String(s)
          .replace(/²/g, "^2")
          .replace(/³/g, "^3")
          .replace(/⁴/g, "^4")
          .replace(/−/g, "-")
          .replace(/,/g, ".")
          .replace(/\s/g, "")
          .replace(/\*/g, "");
      }
      function parse(src) {
        let s = sup(src)
          .replace(/^[a-zA-Z]\(x\)=/, "")
          .replace(/^y=/, "");
        if (!s) throw Error("Bitte eine Funktion eingeben.");
        if (/[^0-9x+\-.^]/.test(s))
          throw Error("Nur Polynome mit x, Zahlen und Potenzen eingeben.");
        s = s.replace(/(?<!^)(?=[+-])/g, "|");
        let terms = s.split("|"),
          coef = [];
        for (let t of terms) {
          if (!t) continue;
          let match = t.match(/^([+-]?)(?:(\d*\.?\d+))?(x(?:\^(\d+))?)?$/);
          if (!match || (!match[2] && !match[3])) throw Error("Ungültiger Term: " + t);
          let sign = match[1] === "-" ? -1 : 1,
            pow = match[3] ? Number(match[4] || 1) : 0,
            c = sign * (match[2] ? Number(match[2]) : 1);
          if (pow > 12 || !Number.isInteger(pow)) throw Error("Maximal Polynomgrad 12.");
          coef[pow] = (coef[pow] || 0) + c;
        }
        while (coef.length > 1 && Math.abs(coef.at(-1)) < 1e-12) coef.pop();
        if (!coef.length) coef = [0];
        return Array.from({ length: coef.length }, (_, i) => coef[i] || 0);
      }
      function evaluate(c, x) {
        let y = 0;
        for (let i = c.length - 1; i >= 0; i--) y = y * x + c[i];
        return y;
      }
      function deriv(c) {
        return c.length <= 1 ? [0] : c.slice(1).map((v, i) => v * (i + 1));
      }
      function fmt(v) {
        if (!Number.isFinite(v)) return "–";
        if (Math.abs(v) < 1e-8) v = 0;
        return Number(v.toFixed(5)).toLocaleString("de-DE", { maximumFractionDigits: 5 });
      }
      function poly(c) {
        let a = [];
        for (let i = c.length - 1; i >= 0; i--) {
          let v = c[i];
          if (Math.abs(v) < 1e-10) continue;
          let s = v < 0 ? "-" : a.length ? "+" : "";
          let n = Math.abs(v);
          a.push(
            s +
              (i && Math.abs(n - 1) < 1e-10 ? "" : fmt(n)) +
              (i ? "x" + (i > 1 ? "^" + i : "") : ""),
          );
        }
        return a.join(" ").replace(/\s([+-])/g, " $1 ") || "0";
```

```
    }
    function bisect(c, a, b) {
      let fa = evaluate(c, a);
      for (let k = 0; k < 90; k++) {
        let m = (a + b) / 2,
          f = evaluate(c, m);
        if (Math.abs(f) < 1e-12 || b - a < 1e-10) return m;
        if (fa * f <= 0) b = m;
        else {
          a = m;
          fa = f;
        }
      }
      return (a + b) / 2;
    }
    function unique(a) {
      return a
        .filter(Number.isFinite)
        .sort((a, b) => a - b)
        .filter((x, i, ar) => i === 0 || Math.abs(x - ar[i - 1]) > 1e-5);
    }
    // Recursive derivative isolation finds real roots, including even-multiplicity roots.
    function roots(c) {
      let n = c.length - 1;
      if (n <= 0) return [];
      if (n === 1) return Math.abs(c[1]) < 1e-12 ? [] : [-c[0] / c[1]];
      let critical = roots(deriv(c));
      let lead = Math.abs(c[n]);
      let bound = 1 + Math.max(...c.slice(0, n).map((v) => Math.abs(v) / lead));
      bound = Math.min(Math.max(bound, 1), 1e7);
      let pts = [-bound, ...critical.filter((x) => x > -bound && x < bound), bound];
      let r = critical.filter(
        (x) => Math.abs(evaluate(c, x)) < 1e-6 * Math.max(1, Math.abs(c[n])),
      );
      for (let i = 0; i < pts.length - 1; i++) {
        let a = pts[i],
          b = pts[i + 1],
          fa = evaluate(c, a),
          fb = evaluate(c, b);
        if (fa * fb < 0) r.push(bisect(c, a, b));
      }
      return unique(r);
    }
    function analysisData(c) {
      let d = deriv(c),
        dd = deriv(d),
        ddd = deriv(dd);
      let zeros = roots(c),
        ext = roots(d).map((x) => ({
          x,
          y: evaluate(c, x),
          kind:
            evaluate(dd, x) > 1e-7
              ? "Tiefpunkt"
              : evaluate(dd, x) < -1e-7
                ? "Hochpunkt"
                : "Stationärer Punkt",
        }));
      let inf = roots(dd)
        .filter((x) => {
          let h = Math.max(1e-4, Math.abs(x) * 1e-5);
          return evaluate(dd, x - h) * evaluate(dd, x + h) < 0;
        })
        .map((x) => ({ x, y: evaluate(c, x) }));
      return { d, dd, ddd, zeros, ext, inf };
    }
    function draw() {
      let rect = canvas.getBoundingClientRect(),
        w = rect.width,
        h = rect.height,
        dpr = devicePixelRatio || 1;
      canvas.width = Math.round(w * dpr);
      canvas.height = Math.round(h * dpr);
      ctx.setTransform(dpr, 0, 0, dpr, 0, 0);
      ctx.fillStyle = "#fff";
      ctx.fillRect(0, 0, w, h);
      let X = (x) => w / 2 + (x - view.cx) * view.scale,
        Y = (y) => h / 2 - (y - view.cy) * view.scale,
```

```
          invX = (p) => (p - w / 2) / view.scale + view.cx,
          invY = (p) => (h / 2 - p) / view.scale + view.cy;
        let step = Math.pow(10, Math.floor(Math.log10(85 / view.scale)));
        if (step * view.scale < 45) step *= 2;
        if (step * view.scale < 45) step *= 2.5;
        let xmin = invX(0),
          xmax = invX(w),
          ymin = invY(h),
          ymax = invY(0);
        ctx.font = "11px system-ui";
        ctx.textBaseline = "middle";
        for (let k = Math.ceil(xmin / step); k <= Math.floor(xmax / step) && k < 1e6; k++) {
          let v = k * step,
            p = X(v);
          ctx.strokeStyle = "#e7e7ed";
          ctx.lineWidth = 1;
          ctx.beginPath();
          ctx.moveTo(p, 0);
          ctx.lineTo(p, h);
          ctx.stroke();
          if (Math.abs(v) > step / 100) {
            ctx.fillStyle = "#606075";
            ctx.fillText(fmt(v), p + 3, Math.min(h - 12, Math.max(12, Y(0) + 12)));
          }
        }
        for (let k = Math.ceil(ymin / step); k <= Math.floor(ymax / step) && k < 1e6; k++) {
          let v = k * step,
            p = Y(v);
          ctx.strokeStyle = "#e7e7ed";
          ctx.lineWidth = 1;
          ctx.beginPath();
          ctx.moveTo(0, p);
          ctx.lineTo(w, p);
          ctx.stroke();
          if (Math.abs(v) > step / 100) {
            ctx.fillStyle = "#606075";
            ctx.fillText(fmt(v), Math.max(4, Math.min(w - 48, X(0) + 6)), p - 7);
          }
        }
        ctx.strokeStyle = "#41415c";
        ctx.lineWidth = 1.7;
        ctx.beginPath();
        ctx.moveTo(X(0), 0);
        ctx.lineTo(X(0), h);
        ctx.moveTo(0, Y(0));
        ctx.lineTo(w, Y(0));
        ctx.stroke();
        ctx.fillStyle = "#302e55";
        ctx.font = "bold 12px system-ui";
        ctx.fillText("Ordinate", Math.min(w - 65, Math.max(5, X(0) + 8)), 13);
        ctx.fillText("Abszisse", w - 69, Math.min(h - 10, Math.max(15, Y(0) - 13)));
        for (let f of funcs) {
          ctx.strokeStyle = f.color;
          ctx.lineWidth = f === funcs[selected] ? 2.8 : 2.2;
          ctx.beginPath();
          let started = false,
            oldY = null;
          for (let px = 0; px <= w; px += 1.5) {
            let py = Y(evaluate(f.c, invX(px)));
            if (
              !Number.isFinite(py) ||
              Math.abs(py) > 1e7 ||
              (oldY !== null && Math.abs(py - oldY) > h * 2)
            ) {
              started = false;
              oldY = py;
              continue;
            }
            if (!started) {
              ctx.moveTo(px, py);
              started = true;
            } else ctx.lineTo(px, py);
            oldY = py;
          }
          ctx.stroke();
          if (!$("markers").checked) continue;
          let a = analysisData(f.c);
          let points = [
```

```
            ...a.zeros.map((x) => ({ x, y: 0, label: "N" })),
            { x: 0, y: f.c[0], label: "Y" },
            ...a.ext.map((p) => ({
              ...p,
              label: p.kind === "Hochpunkt" ? "H" : p.kind === "Tiefpunkt" ? "T" : "S",
            })),
            ...a.inf.map((p) => ({ ...p, label: "W" })),
          ];
          for (let p of points) {
            let px = X(p.x),
              py = Y(p.y);
            if (px < 0 || px > w || py < 0 || py > h) continue;
            ctx.beginPath();
            ctx.fillStyle = "#fff";
            ctx.strokeStyle = f.color;
            ctx.lineWidth = 2;
            ctx.arc(px, py, 4.5, 0, Math.PI * 2);
            ctx.fill();
            ctx.stroke();
            ctx.font = "bold 11px system-ui";
            ctx.fillStyle = f.color;
            ctx.fillText(p.label, px + 7, py - 9);
          }
        }
      }
      function render() {
        let holder = $("functions");
        holder.innerHTML = "";
        funcs.forEach((f, i) => {
          let row = document.createElement("div");
          row.className = "item" + (i === selected ? " selected" : "");
          let dot = document.createElement("span");
          dot.className = "dot";
          dot.style.background = f.color;
          let name = document.createElement("span");
          name.className = "fn";
          name.textContent = String.fromCharCode(102 + i) + "(x) = " + poly(f.c);
          let pick = document.createElement("button");
          pick.className = "small";
          pick.textContent = "Analyse";
          pick.onclick = () => {
            selected = i;
            render();
          };
          let del = document.createElement("button");
          del.className = "small";
          del.textContent = "×";
          del.setAttribute("aria-label", "Funktion löschen");
          del.onclick = () => {
            funcs.splice(i, 1);
            selected = Math.min(selected, funcs.length - 1);
            render();
          };
          row.append(dot, name, pick, del);
          holder.append(row);
        });
        let f = funcs[selected],
          out = $("analysis");
        if (!f) {
          out.innerHTML = "<p>Füge eine Funktion hinzu.</p>";
          draw();
          return;
        }
        let a = analysisData(f.c),
          c = f.c;
        let points = (arr, kind) =>
          arr.length
            ? arr
                .map((p) =>
                  kind === "root"
                    ? "x = " + fmt(p)
                    : "(" + fmt(p.x) + " | " + fmt(p.y) + ")" + (p.kind ? " " + p.kind : ""),
                )
                .join("; ")
            : "Keine reellen Stellen";
        let critical = a.ext.map((p) => p.x),
          bounds = [-Infinity, ...critical, Infinity],
          mon = [];
```

```
    for (let i = 0; i < bounds.length - 1; i++) {
      let lo = bounds[i],
        hi = bounds[i + 1],
        test =
          Number.isFinite(lo) && Number.isFinite(hi)
            ? (lo + hi) / 2
            : Number.isFinite(lo)
              ? lo + Math.max(1, Math.abs(lo) * 0.1)
              : Number.isFinite(hi)
                ? hi - Math.max(1, Math.abs(hi) * 0.1)
                : 0;
      let sign = evaluate(a.d, test);
      mon.push(
        (sign > 1e-8 ? "Steigend" : sign < -1e-8 ? "Fallend" : "Konstant") +
          " auf (" +
          (Number.isFinite(lo) ? fmt(lo) : "−∞") +
          "; " +
          (Number.isFinite(hi) ? fmt(hi) : "+∞") +
          ")",
      );
    }
    out.replaceChildren();
    let sections = [
      ["1. Ausgangsfunktion", "f(x) = " + poly(c)],
      [
        "2. Ableitungen",
        "f′(x) = " + poly(a.d) + "\nf″(x) = " + poly(a.dd) + "\nf‴(x) = " + poly(a.ddd),
      ],
      [
        "3. Achsenschnittpunkte",
        "Nullstellen: f(x) = 0 lösen.\n" +
          points(a.zeros, "root") +
          "\ny-Achsenabschnitt: f(0) = " +
          fmt(c[0]) +
          " → (0 | " +
          fmt(c[0]) +
          ")",
      ],
      [
        "4. Extremstellen",
        "Notwendige Bedingung: f′(x) = 0.\nKandidaten: " +
          points(a.ext, "point") +
          "\nPrüfung mit f″(x): positiv = Tiefpunkt, negativ = Hochpunkt; bei 0 weitere Prüfung erfo
rderlich.",
      ],
      [
        "5. Wendestellen",
        "Notwendige Bedingung: f″(x) = 0.\nVorzeichenwechsel von f″ geprüft.\nWendepunkte: " +
          points(a.inf, "point"),
      ],
      [
        "6. Monotonie",
        "Vorzeichen von f′(x) zwischen kritischen Stellen prüfen.\n" + mon.join("\n"),
      ],
    ];
    for (let [heading, body] of sections) {
      let h = document.createElement("h3");
      h.textContent = heading;
      let p = document.createElement("p");
      p.style.whiteSpace = "pre-line";
      p.textContent = body;
      out.append(h, p);
    }
    draw();
  }
  $("add").onclick = () => {
    try {
      let c = parse($("expr").value);
      funcs.push({ c, color: palette[funcs.length % palette.length] });
      selected = funcs.length - 1;
      $("expr").value = "";
      $("err").textContent = "";
      render();
    } catch (e) {
      $("err").textContent = e.message;
    }
  };
  $("expr").addEventListener("keydown", (e) => {
```

```
        if (e.key === "Enter") $("add").click();
      });
      $("markers").onchange = draw;
      $("zin").onclick = () => {
        view.scale = Math.min(1e5, view.scale * 1.5);
        draw();
      };
      $("zout").onclick = () => {
        view.scale = Math.max(0.001, view.scale / 1.5);
        draw();
      };
      $("reset").onclick = () => {
        view = { cx: 0, cy: 0, scale: 38 };
        draw();
      };
      function distance(a, b) {
        return Math.hypot(a.x - b.x, a.y - b.y);
      }
      canvas.addEventListener("pointerdown", (e) => {
        canvas.setPointerCapture(e.pointerId);
        pointers.set(e.pointerId, { x: e.clientX, y: e.clientY });
        gesture = null;
      });
      canvas.addEventListener("pointermove", (e) => {
        if (!pointers.has(e.pointerId)) return;
        let prev = [...pointers.values()];
        pointers.set(e.pointerId, { x: e.clientX, y: e.clientY });
        let next = [...pointers.values()];
        if (next.length === 1) {
          view.cx -= (next[0].x - prev[0].x) / view.scale;
          view.cy += (next[0].y - prev[0].y) / view.scale;
        } else if (next.length === 2) {
          let before = distance(...prev),
            after = distance(...next);
          if (before > 0) {
            let rect = canvas.getBoundingClientRect(),
              midBefore = {
                x: (prev[0].x + prev[1].x) / 2 - rect.left,
                y: (prev[0].y + prev[1].y) / 2 - rect.top,
              },
              midAfter = {
                x: (next[0].x + next[1].x) / 2 - rect.left,
                y: (next[0].y + next[1].y) / 2 - rect.top,
              };
            let wx = view.cx + (midBefore.x - rect.width / 2) / view.scale,
              wy = view.cy - (midBefore.y - rect.height / 2) / view.scale;
            view.scale = Math.max(0.001, Math.min(1e5, (view.scale * after) / before));
            view.cx = wx - (midAfter.x - rect.width / 2) / view.scale;
            view.cy = wy + (midAfter.y - rect.height / 2) / view.scale;
          }
        }
        draw();
      });
      for (let ev of ["pointerup", "pointercancel", "lostpointercapture"])
        canvas.addEventListener(ev, (e) => pointers.delete(e.pointerId));
      canvas.addEventListener(
        "wheel",
        (e) => {
          e.preventDefault();
          view.scale = Math.max(
            0.001,
            Math.min(1e5, view.scale * (e.deltaY < 0 ? 1.12 : 1 / 1.12)),
          );
          draw();
        },
        { passive: false },
      );
      window.addEventListener("resize", draw);
      funcs = [
        { c: [0, -3, 0, 1], color: palette[0] },
        { c: [-4, 0, 1], color: palette[1] },
      ];
      render();
    </script>
  </body>
</html>
```

6. Build, Test and Modify

6.1 Test the web part in a browser

Because `index.html` is self-contained, it can be opened directly in a desktop browser (double-click, or `python3 -m http.server` in the folder). This is the fastest way to test changes to the user interface or the mathematics. Use the browser developer tools' device mode to check the narrow layout.

6.2 Modify the existing APK

```
apktool d Kurverafx.apk -o kurverafx_src
# edit kurverafx_src/assets/index.html
apktool b kurverafx_src -o Kurverafx_unsigned.apk
zipalign -p 4 Kurverafx_unsigned.apk Kurverafx_aligned.apk
apksigner sign --ks my.keystore --out Kurverafx_signed.apk Kurverafx_aligned.apk
adb install Kurverafx_signed.apk
```

The signature of the original APK cannot be reused: a modified APK must be signed with your own key, and an app installed with a different key has to be uninstalled before the new version can be installed.

6.3 Rebuild as an Android Studio project

- Create a new *Empty Views Activity* project with the package name `de.kurverafx.app`, language Java, minimum SDK 26.
- Copy `source/index.html` to `app/src/main/assets/index.html`.
- Replace the generated activity with `source/MainActivity.java` (the project template may use AppCompat; this activity only needs the platform `Activity`).
- Set the app name to “Kurverafx”, set `targetSdk` to 36 and keep the manifest free of permissions.
- Build a release APK / AAB and sign it with your own key.

6.4 Manual test checklist

Test	Expected result
`x^3 - 3x` added	Roots −1.73205, 0, 1.73205; H(−1 | 2); T(1 | −2); W(0 | 0); increasing – decreasing – increasing
`x^2-4` added	Roots −2 and 2; T(0 | −4); no inflection point
`x^4` added	Root 0; the point (0 | 0) is shown as “Stationärer Punkt” (known limitation)
`f(x)=0,5x^2`	Accepted (prefix and decimal comma); shown as “0,5x^2”
`(x+1)(x-2)`	Error: “Nur Polynome mit x, Zahlen und Potenzen eingeben.”
`x^13`	Error: “Maximal Polynomgrad 12.”
Empty input	Error: “Bitte eine Funktion eingeben.”
Pinch / drag / wheel	Graph zooms around the finger midpoint / moves / zooms; limits 0.001 … 100,000 px per unit
Marker checkbox off	N, Y, H, T, S, W markers disappear
Rotate device	Page reloads with the two example functions (see 4.10)
Delete all functions	Analysis shows “Füge eine Funktion hinzu.”

6.5 Roadmap ideas

- English and other UI languages (all texts are inline strings in the script).

- Classify even-order stationary points (e.g. via the first non-vanishing derivative) instead of reporting them as “stationary”.
- Persist the function list (enable DOM storage or handle configuration changes in the manifest).
- Symmetry, end behaviour, concavity intervals, tangent and normal lines, value table, result export.
- Input of factored forms such as (x+1)(x−2) and rational functions with asymptotes.

Appendix A – Numerical constants

Constant	Value	Used in
Maximum degree	12	parse
Zero-coefficient trim	10^{-12}	parse
Root bound limit	1 ... 10^{7}	roots
Bisection	≤ 90 iterations; stop if width < 10^{-10} or $\lvert f\rvert < 10^{-12}$	bisect
Double-root test	$\lvert f(x)\rvert < 10^{-6}\cdot\max(1, \lvert a_n\rvert)$	roots
De-duplication distance	10^{-5}	unique
Extremum classification	$\lvert f''\rvert > 10^{-7}$	analysisData
Inflection test offset	$h = \max(10^{-4}, \lvert x\rvert\cdot 10^{-5})$	analysisData
Monotonicity threshold	10^{-8}	render
Zero display threshold	10^{-10} (poly), 10^{-8} (fmt)	poly, fmt
Decimals shown	5 (German locale)	fmt
Default view	centre (0, 0), 38 px per unit	view
Zoom limits	0.001 ... 100,000 px per unit	buttons, pinch, wheel
Zoom factors	1.5 (buttons), 1.12 (wheel)	handlers
Curve sampling	every 1.5 px	draw
Grid spacing	target 85 px, minimum 45 px	draw
Marker radius	4.5 px	draw
Canvas height	390 px (≥ 850 px viewport: 560 px)	CSS

Appendix B – License and author

Kurverafx is released under the MIT License (see the file `LICENSE` in the repository). Copyright © 2026 Daniel Kiebler.

created by Daniel Kiebler